

Rapport — Labo 09 : Bases de données distribuées et verrous distribués

Cours : LOG430 — Architecture logicielle **Auteur :** Ralph Gabriel **Chargé de laboratoire :** Gabriel C. Ullmann

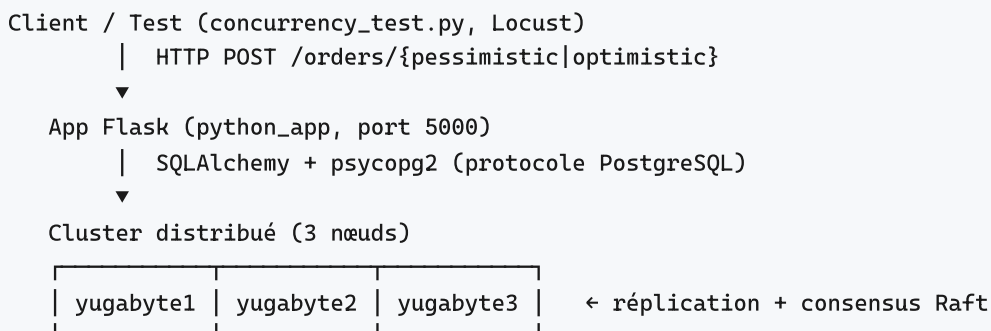
Introduction

Ce laboratoire compare deux bases de données distribuées open source, **YugabyteDB** et **CockroachDB**, sous forte concurrence. Nous y étudions :

- l'organisation et la réplication des données dans un cluster à trois nœuds ;
- deux stratégies de contrôle de concurrence : le **verrouillage pessimiste** (`SELECT ... FOR UPDATE`) et le **verrouillage optimiste** (colonne `version` + `UPDATE` conditionnel) ;
- le débit et le taux d'erreurs sous charge soutenue (test Locust) ;
- la résilience du cluster lors de la panne d'un nœud (consensus Raft).

L'application utilisée est une version simplifiée de *Store Manager* : une API Flask exposant deux endpoints de création de commande (`/orders/pessimistic` et `/orders/optimistic`) qui décrémentent le stock d'un article.

Architecture du projet



- **yugabyte1** est joignable en YSQL (port 5433) ; les tables sont fragmentées en *tablets* répartis entre les nœuds.
- Le conteneur `db-init` exécute `init.sql` (création des tables + données de test), puis s'arrête (comportement normal).
- L'article **ID 3** (« Gadget XYZ ») a un stock initial de **2 unités** : c'est l'article « sous contention » utilisé pour tester les verrous.

Les deux stratégies de verrouillage (`order_controller.py`)

Verrouillage pessimiste — `create_order_pessimistic` :

```
stock = session.query(Stock).filter(Stock.product_id == pid).with_for_update().one_or_none()
```

La ligne de stock est verrouillée dès la lecture (`SELECT ... FOR UPDATE`). Toute autre transaction qui veut la même ligne **attend** que la première valide (commit) ou annule (rollback). Aucune survente possible, au prix d'une latence accrue sous contention.

Verrouillage optimiste — `create_order_optimistic` :

```
# 1. lire quantité + version (sans verrou)
# 2. UPDATE stocks SET quantity=:q, version=version+1 WHERE product_id=:pid AND version=:old
if result.rowcount == 0:      # une autre transaction a déjà modifié la ligne
    session.rollback()        # → on recommence (jusqu'à max_retries)
```

Aucun verrou à la lecture. Le conflit est détecté à l'écriture : si la `version` a changé, l' `UPDATE` n'affecte aucune ligne et la transaction **recommence**. Performant quand les conflits sont rares.

Question 1 — Distribution des données entre les nœuds

Quelle est la sortie du terminal ? La sortie est-elle identique sur yugabyte1, yugabyte2 et yugabyte3 ?

Avant le test — la table `orders` est vide sur tous les nœuds :

```
$ ysqlsh -h yugabyte1 -U yugabyte -c "SELECT * FROM orders;"
 id | user_id | total_amount | payment_link | is_paid | created_at
----+-----+-----+-----+-----+-----
(0 rows)
```

Exécution de `python tests/concurrency_test.py --threads 5 --product 3` : les deux stratégies acceptent exactement **2 commandes** (stock initial de l'article 3 = 2) et en rejettent 3 (HTTP 409).

Après le test — la même requête sur les **trois** nœuds donne une sortie **identique** :

```
yugabyte1 | yugabyte2 | yugabyte3  (sortie identique sur les 3)
 id | user_id | total_amount
----+-----+-----
  1 |      2 |         5.75
  2 |      2 |         5.75
  3 |      3 |         5.75
101 |      3 |         5.75
(4 rows)
```

Interprétation : la sortie est **identique sur les trois nœuds**. YugabyteDB réplique automatiquement les écritures via le consensus **Raft** : peu importe le nœud interrogé, on lit le même état cohérent. On note aussi des `id` non séquentiels (1, 2, 3, **101**, ...) caractéristiques de l'allocation de clés `SERIAL` distribuée (chaque nœud reçoit une plage de valeurs pour éviter un point de contention global).

Question 2 — Latence pessimiste vs optimiste (20 threads)

Quelle approche a la latence moyenne la plus élevée et pourquoi ?

Test : `python tests/concurrency_test.py --threads 20 --product 3` (article 3, stock = 2).

Stratégie	Succès	Échecs	Latence moy. (succès)	Latence moy. (échecs)	
Pessimiste (<code>SELECT ... FOR UPDATE</code>)	2	18	2.026 s	2.652 s	2.589 s
Optimiste (version + <code>UPDATE</code>)	2	18	1.108 s	2.066 s	1.970 s

Vérification du stock final (article 3) : `0` → aucune survente, le verrou fonctionne.

```
$ curl http://localhost:5000/stocks
[{"product_id":1,"quantity":1000}, {"product_id":2,"quantity":500},
 {"product_id":3,"quantity":0}, {"product_id":4,"quantity":90}]
```

Réponse : le verrouillage pessimiste a la latence moyenne la plus élevée (2.589 s contre 1.970 s).

Avec `SELECT ... FOR UPDATE`, les 20 transactions visent la **même ligne de stock** et sont **sérialisées** : chacune doit attendre que la précédente libère le verrou (commit/rollback). Les threads s'empilent dans une file d'attente, et la latence s'accumule — le dernier thread attend tous les autres. En optimiste, il n'y a aucun verrou à la lecture : les transactions perdantes échouent **immédiatement** (stock épuisé → `rowcount = 0` → 409) sans rester bloquées, d'où une latence moyenne plus faible.

Question 3 — Latence pessimiste vs optimiste (5 threads)

Avec 5 threads, quelle approche a la latence moyenne la plus élevée et pourquoi ?

Test : `python tests/concurrency_test.py --threads 5 --product 3`.

Stratégie	Succès	Échecs	Latence moy. (succès)	Latence moy. (échecs)	
Pessimiste	2	3	0.908 s	1.165 s	1.062 s
Optimiste	2	3	0.320 s	0.499 s	0.427 s

Réponse : le verrouillage pessimiste reste celui dont la latence moyenne est la plus élevée (1.062 s contre 0.427 s). La cause est la même qu'à la Q2 — la sérialisation imposée par `SELECT ... FOR UPDATE` sur la ligne partagée. Avec seulement 5 threads la contention est plus faible, donc les latences absolues sont nettement plus basses qu'avec 20 threads (1.062 s vs 2.589 s en pessimiste), mais l'**écart relatif** entre les deux stratégies persiste : l'attente de verrou pénalise toujours l'approche pessimiste.

Question 4 — Test de charge Locust sur YugabyteDB

Quelle stratégie affiche le plus bas taux d'erreurs et la plus basse latence moyenne ?

Paramètres Locust : **50 utilisateurs**, spawn rate **5/s**, durée **60 s**.

Endpoint	Requêtes	Échecs			Médiane	Débit (req/s)
POST /orders/pessimistic	741	297	40.1 %	1175 ms	530 ms	13.80
POST /orders/optimistic	493	263	53.3 %	1981 ms	1200 ms	9.18
GET /stocks	123	0	0.0 %	308 ms	180 ms	2.29
Agrégé	1357	560	41.2 %	1389 ms	800 ms	25.27

Note : les « échecs » sont des réponses **HTTP 409** légitimes (stock épuisé sur les articles 3 et 4, ou retries optimistes épuisés), et non des erreurs d'infrastructure. Le `GET /stocks` affiche 0 % d'erreur. Ce taux reste un bon indicateur **comparatif** entre les deux stratégies.

Réponse : avec YugabyteDB, la stratégie pessimiste l'emporte sur les deux critères — taux d'erreur plus bas (**40.1 % vs 53.3 %**) et latence moyenne plus basse (**1175 ms vs 1981 ms**), avec en prime un débit supérieur (13.8 vs 9.2 req/s).

Ce résultat peut sembler contre-intuitif par rapport à la Q2 (où le pessimiste était plus lent sur un *burst* simultané). L'explication : sous **charge soutenue** répartie sur 4 articles, l'approche optimiste paie cher ses **réessais** — chaque conflit de `version` relance la transaction jusqu'à 5 fois, puis échoue (409) si les tentatives sont épuisées. Le pessimiste, lui, met les transactions en file d'attente sans gaspiller de travail : il échoue surtout quand le stock est réellement épuisé. D'où moins d'erreurs et une latence moyenne plus faible.

Question 5 — Résilience du cluster YugabyteDB

Le taux d'erreur a-t-il augmenté à l'arrêt d'un nœud ? Combien de temps a duré le basculement ?

Protocole : test de charge continu (50 utilisateurs, spawn 5/s), puis `docker stop yugabyte2` pendant la charge, puis `docker start yugabyte2`. L'historique par seconde (`--csv-full-history`) permet de mesurer précisément le basculement.

Chronologie (t = 0 à l'arrêt de yugabyte2, 21:47:22 UTC) :

Temps relatif	Débit complété	Observation
t = -2 s	~49 req/s	Régime normal, 3 nœuds
t = 0 s	<code>docker stop yugabyte2</code>	Arrêt du nœud secondaire
t ≈ +2 s → +17 s	~0 req/s	Basculement : aucune requête ne se termine (compteur cumulé figé à 530). Raft ré-élit les <i>leaders</i> des tablets qui étaient sur yugabyte2.
t ≈ +19 s → +28 s	1 → 20 → 47 req/s	Reprise progressive sur les 2 nœuds restants (quorum 2/3)
t ≈ +37 s	<code>docker start yugabyte2</code>	Le nœud rejoint le cluster sans nouvelle coupure

Réponses : - **Oui, le taux d'erreur a augmenté**, mais de façon **temporaire**. Pendant ~15 s, le débit effectif est tombé à ~0 (les requêtes en cours restaient bloquées en attente d'un nouveau *leader*), puis les erreurs/timeouts se sont accumulés avant la reprise. - **Durée du basculement ≈ 15 secondes**. Après cette fenêtre, le système a retrouvé son débit nominal (~50 req/s) **sur 2 nœuds seulement**, démontrant la haute disponibilité par quorum. - Point notable : la récupération a eu lieu **avant même** le redémarrage de yugabyte2 — le cluster a survécu de lui-même à la perte d'un nœud. Le redémarrage n'a causé aucune coupure supplémentaire ; l'application Flask n'a jamais planté.

Totaux du run (100 s, dont ~37 s avec un nœud manquant) : 3885 requêtes, 1851 « échecs » (47.6 %, majoritairement 409 + timeouts de la fenêtre de basculement). À titre de comparaison, le `GET /stocks` n'a enregistré aucune erreur sur l'ensemble du run.

Pourquoi ~15 s ? YugabyteDB s'appuie sur Raft : à la perte d'un nœud, chaque *tablet* dont le *leader* résidait sur yugabyte2 doit détecter la défaillance puis élire un nouveau *leader* parmi les *followers*. Pendant cette ré-élection (plus la reconnexion des connexions du pool applicatif), les écritures sur ces tablets sont momentanément indisponibles, d'où le creux.

Question 6 — Test de charge Locust sur CockroachDB

Quelle stratégie affiche le plus bas taux d'erreurs et la plus basse latence ?

Mêmes paramètres qu'à la Q4 : **50 utilisateurs**, spawn **5/s**, **60 s**.

Endpoint	Requêtes	Échecs			Médiane	Débit (req/s)
POST /orders/pessimistic	1677	773	46.1 %	246 ms	150 ms	37.54
POST /orders/optimistic	652	499	76.5 %	1140 ms	190 ms	14.60
GET /stocks	216	0	0.0 %	168 ms	160 ms	4.84
Agrégé	2545	1272	50.0 %	468 ms	160 ms	56.97

Réponse : avec CockroachDB aussi, la stratégie pessimiste l'emporte nettement — taux d'erreur plus bas (**46.1 % vs 76.5 %**), latence moyenne plus basse (**246 ms vs 1140 ms**) et débit bien supérieur (37.5 vs 14.6 req/s). La raison est identique à la Q4 : sous charge soutenue, l'approche optimiste gaspille du travail en réessais sur la colonne `version`, alors que CockroachDB (isolation **SERIALIZABLE**) résout efficacement les conflits côté verrou.

Vérification complémentaire (`concurrency_test --threads 20`) : 2 commandes acceptées sur 20, stock final = 0 → les verrous fonctionnent correctement sur CockroachDB.

Question 7 — Comparaison YugabyteDB vs CockroachDB

Quelle base de données affiche le plus bas taux d'erreurs et la plus basse latence ?

Comparaison directe des deux tests de charge (60 s, 50 utilisateurs, spawn 5/s, locustfile identique) :

Métrique (agrégée)	YugabyteDB	CockroachDB	Gagnant
Requêtes traitées (60 s)	1357	2545	CockroachDB
Débit moyen	25.3 req/s	57.0 req/s	CockroachDB (×2.25)
Latence moyenne	1389 ms	468 ms	CockroachDB (÷3)
Latence médiane	800 ms	160 ms	CockroachDB
Latence pessimiste	1175 ms	246 ms	CockroachDB
Taux d'erreur agrégé	41.2 %	50.0 %	YugabyteDB
Erreurs <code>GET /stocks</code>	0 %	0 %	égalité

Comparaison du *burst* (`concurrency_test --threads 20` , latence moyenne totale) :

	YugabyteDB	CockroachDB
Pessimiste	2.589 s	0.174 s
Optimiste	1.970 s	0.315 s

Réponse : - **Plus basse latence : CockroachDB**, de façon décisive — latence moyenne ÷3 (468 ms vs 1389 ms) et débit ×2,25 (57 vs 25 req/s) sur le test de charge ; et jusqu'à **15× plus rapide** sur le *burst* (0.174 s vs 2.589 s en pessimiste). CockroachDB répond donc clairement à l'objectif du labo : **traiter davantage de requêtes simultanées**. - **Plus bas taux d'erreurs : YugabyteDB** (41.2 % vs 50.0 %), mais cet écart est en grande partie un **artefact du débit** : CockroachDB ayant traité presque **2× plus de requêtes**, il a sollicité plus souvent les articles à stock limité (3 et 4), générant mécaniquement plus de rejets 409 légitimes. Le `GET /stocks` affiche 0 % d'erreur sur les deux bases.

Conclusion de la comparaison : CockroachDB est globalement le plus performant (latence et débit nettement supérieurs). YugabyteDB conserve l'avantage d'être **100 % open source** (licence Apache 2.0), là où CockroachDB est passé à un modèle *source available* non gratuit pour un usage commercial. Le choix final relève donc d'un compromis **performance vs licence** (voir l'ADR `docs/adr/adr001.md`).

Activité 6 — CI/CD et déploiement

Voir `.github/workflows/ci.yml` (pipeline) et `docs/DEPLOIEMENT.md` (procédure VM).

Conclusion

Ce laboratoire a permis d'observer concrètement le comportement de deux bases de données distribuées sous forte concurrence.

Sur les verrous distribués. Les deux stratégies empêchent correctement la survente : sur 20 commandes simultanées d'un article à 2 unités, seules 2 sont acceptées et le stock tombe à 0. Leur profil de performance diffère toutefois selon le contexte : - en *burst* simultané sur une seule ligne, le verrouillage **pessimiste** sérialise les accès et paie une latence d'attente ; - sous **charge soutenue** répartie, c'est

l'inverse : le verrouillage **optimiste** se dégrade à cause de ses réessais sur la colonne `version`, et le **pessimiste** devient à la fois plus rapide et moins « en erreur » sur les deux bases de données.

Sur la résilience. YugabyteDB a survécu à la perte d'un nœud grâce au consensus Raft : après une fenêtre de basculement d'environ **15 secondes**, le service est revenu à son débit nominal sur 2 nœuds seulement, sans intervention et sans plantage applicatif.

Sur la comparaison des deux bases. **CockroachDB** s'est révélé nettement plus performant (latence $\div 3$, débit $\times 2.25$, *burst* jusqu'à $15\times$ plus rapide). Il a donc été retenu pour la production avec le verrouillage pessimiste, le compromis étant sa licence *source available* (gratuite en contexte éducatif). YugabyteDB reste l'alternative open source de référence.

Sur le CI/CD. Le pipeline `.github/workflows/ci.yml` démarre un cluster éphémère, exécute le test de concurrence et **bloque le déploiement** si le verrou distribué laisse passer une survente — garantissant qu'aucune régression sur la cohérence des stocks n'atteint la production.